影像處理
--CH4_2

姓名:邱榮誠

學號:314512065

系級:電控碩 1

日期:2025/11/03

Q1.Please use FFT and design a frequency filter to cancel the sinusoidal noise of the assigned image, 'astronaut-interference.tif', and print out the source code and the processed image?

1）**source code**:

```
import os
import sys
import cv2 as cv
import numpy as np

def read_gray(path: str) -> np.ndarray:
    img = cv.imread(path, cv.IMREAD_GRAYSCALE)
    return img.astype(np.float32) / 255.0

def write_u8(path: str, img01: np.ndarray):
    cv.imwrite(path, (np.clip(img01, 0, 1) * 255.0).astype(np.uint8))

def dft_shifted(img01: np.ndarray) -> np.ndarray:
    dft = cv.dft(img01, flags=cv.DFT_COMPLEX_OUTPUT)
    return np.fft.fftshift(dft, axes=(0, 1))

def idft_from_shift(dft_shift: np.ndarray) -> np.ndarray:
    ishift = np.fft.ifftshift(dft_shift, axes=(0, 1))
    img_back = cv.idft(ishift, flags=cv.DFT_SCALE | cv.DFT_REAL_OUTPUT)
    mn, mx = float(img_back.min()), float(img_back.max())
    if mx > mn:
        img_back = (img_back - mn) / (mx - mn + 1e-12)
    else:
        img_back = np.clip(img_back, 0.0, 1.0)
    return img_back

def spectrum_log01(dft_shift: np.ndarray) -> np.ndarray:
    mag = cv.magnitude(dft_shift[...,0], dft_shift[...,1])
    mag = np.log1p(mag)
    return cv.normalize(mag, None, 0, 1, cv.NORM_MINMAX)

def auto_detect_peaks(dft_shift: np.ndarray,
```

```
                    K: int = 8,
                    center_exclusion: float = 0.05,
                    cross_w: int = 6,
                    border_ratio: float = 0.02,
                    min_sep: float = 12.0):
    H, W, _ = dft_shift.shape
    cy, cx = H // 2, W // 2
    yv, xv = np.ogrid[:H, :W]
    mag = cv.magnitude(dft_shift[...,0], dft_shift[...,1])
    R = np.sqrt((yv - cy)**2 + (xv - cx)**2)
    mask = np.ones((H, W), dtype=bool)
    mask &= R > (min(H, W) * center_exclusion)
    b = int(min(H, W) * border_ratio)
    mask[:b, :] = False; mask[-b:, :] = False; mask[:, :b] = False;
mask[:, -b:] = False
    mask &= (np.abs(xv - cx) > cross_w) & (np.abs(yv - cy) > cross_w)
    mag_mask = mag.copy()
    mag_mask[~mask] = 0.0
    dist = R / (R.max() + 1e-6)
    score = mag_mask * (1.0 + dist)
    flat = score.ravel()
    K = min(K, flat.size)
    idx = np.argpartition(flat, -K)[-K:]
    coords = np.column_stack(np.unravel_index(idx, score.shape))
    selected = []
    for (py, px) in coords:
        if all((py - sy)**2 + (px - sx)**2 >= min_sep**2 for (sy, sx) in
selected):
            selected.append((int(py), int(px)))
    return selected[: max(1, K // 2)]

def gaussian_notch_mask(shape, peaks, sigma=10.0):
    H, W = shape
    cy, cx = H // 2, W // 2
    yv, xv = np.ogrid[:H, :W]
    mask = np.ones((H, W), dtype=np.float32)
    for (y0, x0) in peaks:
        d2 = (yv - y0)**2 + (xv - x0)**2
```

```
        notch = 1.0 - np.exp(-d2 / (2.0 * sigma * sigma))
        mask *= notch
        y1 = (2 * cy - y0) % H
        x1 = (2 * cx - x0) % W
        d2s = (yv - y1)**2 + (xv - x1)**2
        notch_s = 1.0 - np.exp(-d2s / (2.0 * sigma * sigma))
        mask *= notch_s
    return mask

def apply_mask_complex(dft_shift: np.ndarray, mask: np.ndarray) ->
np.ndarray:
    out = dft_shift.copy()
    out[...,0] *= mask
    out[...,1] *= mask
    return out

def main():
    in_path = "astronaut-interference.tif" if len(sys.argv) < 2 else
sys.argv[1]
    stem, _ = os.path.splitext(in_path)
    img = read_gray(in_path)
    dft_s = dft_shifted(img)
    spec_before = spectrum_log01(dft_s)
    write_u8(f"{stem}_spectrum.png", spec_before)
    peaks = auto_detect_peaks(
        dft_s,
        K=8,
        center_exclusion=0.05,
        cross_w=6,
        border_ratio=0.02,
        min_sep=14.0
    )
    print("Detected peaks (y, x):", peaks)
    sigma = 15.0
    mask = gaussian_notch_mask(img.shape, peaks, sigma=sigma)
    dft_f = apply_mask_complex(dft_s, mask)
    spec_after = spectrum_log01(dft_f)
    write_u8(f"{stem}_spectrum_filtered.png", spec_after)
```

```
    img_out = idft_from_shift(dft_f)
    write_u8(f"{stem}_denoised.png", img_out)
    write_u8(f"{stem}_mask.png", mask)

if __name__ == "__main__":
    main()
```

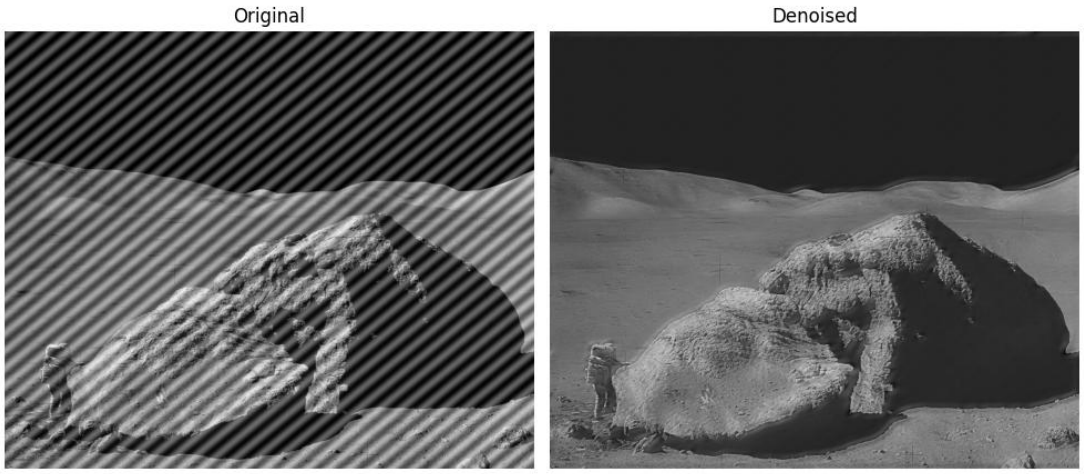
2) **frequency filter-Gaussian Notch-Reject Filter**

對每一個偵測到的頻譜峰值座標(y0,x0)，建立一個以該點為中心的高斯形狀抑制區域，其濾波函數可表示為：

$$H(u,v) = 1 - e^{-\frac{D(u,v)^2}{2\sigma^2}}$$

D(u,v)為頻譜座標(u,v)與干擾峰值點之距離，σ 為控制缺口寬度的參數。本次實驗的目標是消除影像中的斜向干涉條紋，因此在偵測到的斜向亮點位置建立相對應的高斯缺口。由公式可知，當 D(u,v)=0（即位於干擾峰中心）時，H=0，代表該頻率能量被完全抑制；而當距離干擾點愈遠時，H≈1，則表示正常的影像頻率成分能被保留。此外，為確保逆傅立葉轉換後的影像仍為實數，濾波器需同時在亮點及其共軛對稱位置建立相同的高斯缺口，可同步抑制正負頻率能量，達到平衡濾波效果。

3) **processed image**



Q2.Please use FFT and design a frequency filter to cancel the moire-pattern noise of the assigned image, 'car-moire-pattern.tif', and print out the source code and the processed image?

3）**source code**:

```
import cv2 as cv
import numpy as np
import os
import sys
import matplotlib.pyplot as plt

def read_gray(path: str) -> np.ndarray:
    img = cv.imread(path, cv.IMREAD_GRAYSCALE)
    return img.astype(np.float32) / 255.0

def fft_shift(img01: np.ndarray) -> np.ndarray:
    dft = cv.dft(img01, flags=cv.DFT_COMPLEX_OUTPUT)
    return np.fft.fftshift(dft, axes=(0, 1))

def ifft_shift_to_img(dft_shift: np.ndarray) -> np.ndarray:
    ishift = np.fft.ifftshift(dft_shift, axes=(0, 1))
    img_back = cv.idft(ishift, flags=cv.DFT_SCALE | cv.DFT_REAL_OUTPUT)
    img_back = np.clip(img_back, 0, 1)
    return img_back

def log_spectrum(dft_shift: np.ndarray) -> np.ndarray:
    mag = cv.magnitude(dft_shift[...,0], dft_shift[...,1])
    mag = np.log1p(mag)
    mag_norm = cv.normalize(mag, None, 0, 1, cv.NORM_MINMAX)
    return mag_norm

def detect_peaks(dft_shift: np.ndarray, K=8, center_exclusion=0.05,
cross_w=6, border_ratio=0.02):
    H, W, _ = dft_shift.shape
    cy, cx = H // 2, W // 2
    yv, xv = np.ogrid[:H, :W]
    mag = cv.magnitude(dft_shift[...,0], dft_shift[...,1])
```

```
    R = np.sqrt((yv - cy)**2 + (xv - cx)**2)
    mask = np.ones((H, W), dtype=bool)
    mask &= R > (min(H, W) * center_exclusion)
    b = int(min(H, W) * border_ratio)
    mask[:b,:] = mask[-b:,:] = mask[:,:b] = mask[:,-b:] = False
    mask &= (np.abs(xv - cx) > cross_w) & (np.abs(yv - cy) > cross_w)
    mag_mask = mag.copy()
    mag_mask[~mask] = 0.0
    dist = R / (R.max() + 1e-6)
    score = mag_mask * (1.0 + dist)
    idx = np.argpartition(score.ravel(), -K)[-K:]
    coords = np.column_stack(np.unravel_index(idx, score.shape))
    selected = []
    min_sep = 12.0
    for (py, px) in coords:
        if all((py - sy)**2 + (px - sx)**2 >= min_sep**2 for (sy,sx) in
selected):
            selected.append((int(py), int(px)))
    return selected[:max(1, K//2)]

def gaussian_notch_mask(shape, peaks, sigma=10.0):
    H, W = shape
    cy, cx = H // 2, W // 2
    yv, xv = np.ogrid[:H, :W]
    mask = np.ones((H, W), dtype=np.float32)

    for (y0, x0) in peaks:
        d2 = (yv - y0)**2 + (xv - x0)**2
        notch = 1.0 - np.exp(-d2 / (2.0 * sigma**2))
        mask *= notch
        y1 = (2*cy - y0) % H
        x1 = (2*cx - x0) % W
        d2s = (yv - y1)**2 + (xv - x1)**2
        notch_s = 1.0 - np.exp(-d2s / (2.0 * sigma**2))
        mask *= notch_s
    return mask
```

```python
def apply_mask_complex(dft_shift: np.ndarray, mask: np.ndarray) ->
np.ndarray:
    out = dft_shift.copy()
    out[...,0] *= mask
    out[...,1] *= mask
    return out

def main():
    in_path = "car-moire-pattern.tif" if len(sys.argv) < 2 else
sys.argv[1]
    stem, _ = os.path.splitext(in_path)
    img = read_gray(in_path)
    dft_s = fft_shift(img)
    spec_before = log_spectrum(dft_s)
    plt.imsave(f"{stem}_spectrum.png", spec_before, cmap='gray')
    peaks = detect_peaks(dft_s, K=8, center_exclusion=0.05)
    print("Detected peaks (y,x):", peaks)
    sigma = 12.0
    mask = gaussian_notch_mask(img.shape, peaks, sigma=sigma)
    dft_f = apply_mask_complex(dft_s, mask)
    spec_after = log_spectrum(dft_f)
    plt.imsave(f"{stem}_spectrum_filtered.png", spec_after, cmap='gray')
    plt.imsave(f"{stem}_mask.png", mask, cmap='gray')
    img_out = ifft_shift_to_img(dft_f)
    plt.imsave(f"{stem}_denoised.png", img_out, cmap='gray')
    plt.figure(figsize=(10,6))
    plt.subplot(1,2,1); plt.title("Original");
    plt.imshow(img, cmap='gray'); plt.axis('off')
    plt.subplot(1,2,2); plt.title("Denoised");
    plt.imshow(img_out, cmap='gray'); plt.axis('off')
    plt.tight_layout(); plt.show()

if __name__ == "__main__":
    main()
```

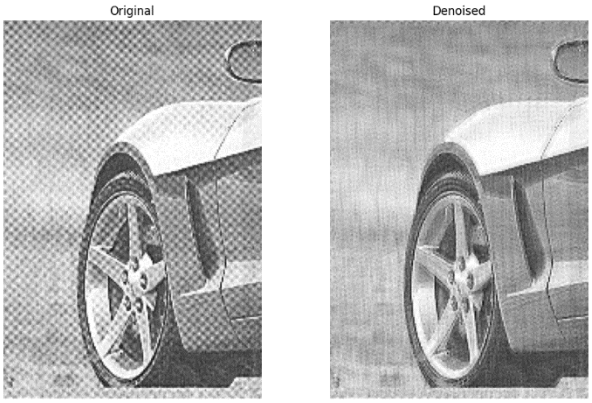
2) frequency filter-Gaussian Notch-Reject Filter

對每一個偵測到的頻譜峰值座標(y0,x0)，建立一個以該點為中心的高斯形狀抑制區域，其濾波函數可表示為：

$$H(u,v) = 1 - e^{-\frac{D(u,v)^2}{2\sigma^2}}$$

D(u,v)為頻譜座標(u,v)與干擾峰值點之距離，σ 為控制缺口寬度的參數。本次實驗的目標是消除影像中的斜向干涉條紋，因此在偵測到的斜向亮點位置建立相對應的高斯缺口。由公式可知，當 D(u,v)=0（即位於干擾峰中心）時，H=0，代表該頻率能量被完全抑制；而當距離干擾點愈遠時，H≈1，則表示正常的影像頻率成分能被保留。此外，為確保逆傅立葉轉換後的影像仍為實數，濾波器需同時在亮點及其共軛對稱位置建立相同的高斯缺口，可同步抑制正負頻率能量，達到平衡濾波效果。

3）processed image



Q3.Please comment and compare your two design freq. filters?

在 astronaut-interference.tif 影像中，干擾屬於方向明確的單一頻率條紋噪聲，其頻譜中出現一對對稱點(Figure1 所示)，對應至干擾頻率與其共軛頻率。因此濾波器採用雙點高斯缺口，僅在這兩個明確位置建立圓形高斯缺口，以平滑方式抑制該特定頻率能量。由於干擾頻率單一，濾波範圍屬於窄頻抑制，能有效去除條紋而不影響原始影像細節。

相較之下，car-moire-pattern.tif 影像的干擾為多方向、多頻率的摩爾紋，其頻譜中呈現多組對稱點分佈於水平方向與斜向(Figure2 所示)。針對此類複雜干擾，濾波器設計為多點高斯缺口，需在多個不同方向的點上建立高斯缺口，分別抑制多組干擾頻率。此設計屬於廣頻抑制，能顯著降低摩爾紋效果，但由於同時抑制多個頻率成分，影像細節會略為平滑。

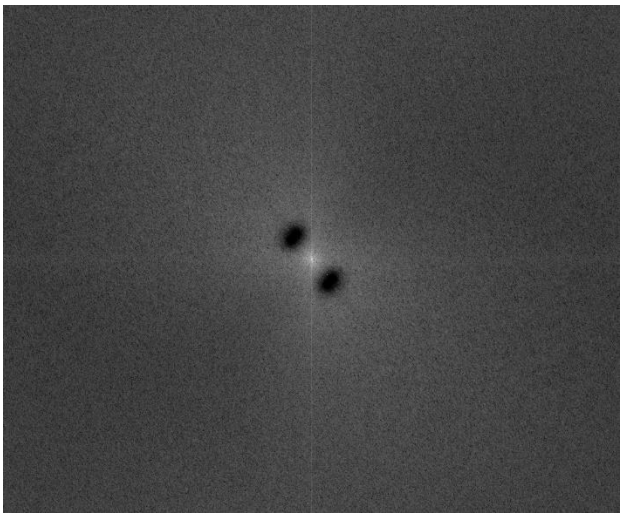


Figure1. astronaut-interference.tif 頻譜對稱點

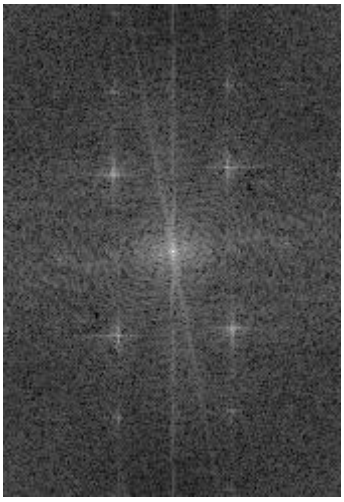


Figure2. car-moire-pattern.tif 頻譜對稱點